

>_ ./start_journey.sh

The Developer's Journey: 8주간의 파이썬 성장기와 비밀번호 생성기 프로젝트

기초 문법부터 GUI 응용 프로그램 완성까지의 기록

Author: 2026964010 김승진

Repository: <https://github.com/2026964010/Python>



**2026964010**

Add files via upload

d3c7dc9 · yesterday

**23 Commits**

0306/resorce

Add files via upload

3 months ago



0320

Add files via upload

3 months ago



0327

Add files via upload

2 months ago



0410

Add files via upload

2 months ago



0417

Add files via upload

2 months ago



0508

Add files via upload

yesterday



0515

Add files via upload

yesterday



3013/src

Add files via upload

3 months ago



0306.md

Add files via upload

yesterday



0313.md

Add files via upload

yesterday



0320.md

Add files via upload

yesterday



0327.md

Add files via upload

yesterday



0410.md

Add files via upload

yesterday



0417.md

Add files via upload

yesterday



0508.md

Add files via upload

yesterday



0515.md

Add files via upload

yesterday



SourceTree-Git (1).pptx

Add files via upload

2 months ago



SourceTree_Simplified.pptx

Add files via upload

2 months ago



hello.py

Add files via upload

3 months ago



readme.md

Create readme.md

3 months ago

<

0306.md

...

print 에 대해 배웠어요. 파이썬 코딩에 대해 기초적이고 중요한 틀을 알게되었어요.

<

0417.md

...

리스트(List) 자료구조의 이해와 활용 다량의 데이터를 체계적으로 관리하고 순차적으로 접근할 수 있는 대표적인 자료구조인 리스트(List)의 개념을 학습하였습니다.

리스트의 생성, 인덱싱(Indexing) 및 슬라이싱(Slicing)을 통한 데이터 추출뿐만 아니라, 다양한 내장 메소드를 활용하여 요소를 추가, 삭제, 정렬하는 동적 데이터 관리 방식을 습득하였습니다.

앞서 학습한 반복문(for 문)과 리스트를 유기적으로 결합하여, 대량의 데이터를 효율적으로 순회하고 처리하는 실습을 진행함으로써 실무적인 데이터 핸들링 능력을 배양하였습니다.

<

0515.md

...

클래스(Class)와 객체 지향 프로그래밍(OOP)의 기초 객체 지향 프로그래밍의 핵심이자 실현 도구인 클래스(Class)와 객체(Object)의 개념을 정의하고 이해하였습니다.

속성(Attribute)을 표현하는 변수와 행위(Behavior)를 정의하는 메서드(Method)를 하나의 단위로 묶는 캡슐화 과정을 실습하였습니다.

생성자 메서드(init)를 통해 객체를 초기화하고 인스턴스를 생성 및 활용하는 방법을 익힘으로써, 복잡한 현실 세계의 데이터를 소프트웨어적으로 구조화하고 대규모 프로그램의 확장성과 유지보수성을 높이는 설계 기법의 초석을 다졌습니다.

<

0313.md

...

GitHub 저장소 README.md 파일에 정중하고 격식 있게 작성할 수 있도록 구성한 서식입니다. 필요에 따라 프로젝트명이나 세부 내용을 수정하여 사용하시기 바랍니다.

본 저장소는 프로그래밍의 견고한 기초를 다지기 위해 실습하고 학습한 내용을 기록하는 공간입니다.

기초가 튼튼한 개발자로 성장하기 위한 첫걸음으로, 가장 기본이 되는 핵심 개념들을 단계별로 파악하고 구현해 보는 것에 초점을 맞추어 진행하였습니다.

변수(Variables)의 이해와 활용 데이터의 저장 및 관리를 위한 메모리 공간의 개념을 이해하고, 다양한 데이터 타입에 따른 변수 선언과 할당 과정을 학습하였습니다.

변수의 유효 범위(Scope)와 생명 주기(Lifecycle)를 고려하여 코드의 안정성을 높이는 데이터 관리 방법을 익혔습니다.

문자열(Strings) 데이터 처리 텍스트 데이터를 효율적으로 다루기 위한 문자열 선언, 결합, 그리고 다양한 내장 메소드(Method)의 활용법을 습득하였습니다.

포매팅(Formatting) 기법을 통해 가독성 높은 문자열 출력을 구현하고, 동적인 데이터 표현 방식을 실습하였습니다.

<

0320.md

...

제어문(Control Flow Statements)의 이해와 활용 프로그램의 실행 흐름을 제어하는 핵심 개념인 조건문(Conditional Statements)과 반복문(Loop Statements)의 구조와 원리를 학습하였습니다.

for 문과 while 문을 활용하여 반복적인 작업을 효율적으로 처리하고, 특정 조건에 따라 코드의 실행 여부를 결정하는 분기 로직을 구현하였습니다.

복잡한 조건을 논리적으로 구조화하여 코드의 가독성을 높이고, 프로그램의 동적인 제어 흐름을 설계하는 능력을 배양하였습니다.

<

0410.md

...

함수(Functions)의 정의와 반환(Return) 처리 코드의 재사용성과 모듈화를 위한 핵심 개념인 함수 정의(def)와 결과값 반환(return)의 메커니즘을 학습하였습니다.

입력값(Parameter/Argument)을 받아 정해진 로직을 수행하고, 처리된 결과를 호출부로 안전하게 전달하는 데이터 흐름을 이해하였습니다.

중복되는 코드를 최소화하고 기능별로 로직을 분리함으로써, 전체 프로그램의 가독성과 유지보수성을 크게 향상시키는 구조적 프로그래밍을 실습하였습니다.

<

0327.md

...

조건문과 제어문의 종합적 활용 프로그램의 논리적 흐름을 결정하는 조건문(Conditional Statements)과 실행을 제어하는 제어문(Control Flow Statements)의 유기적인 연계를 학습하였습니다.

다양한 조건을 분석하여 프로그램이 상황에 따라 유연하게 대처할 수 있도록 분기 처리를 정교화하고, 무분별한 반복을 방지하는 효율적인 로직 제어를 실습하였습니다.

복잡한 요구사항을 단순하고 직관적인 논리 구조로 변환하는 과정을 통해, 문제 해결 능력과 코드의 최적화 가능성을 넓혔습니다.

<

0508.md

...

튜플(Tuple) 자료구조의 특징과 활용 데이터를 순차적으로 저장하는 선형 자료구조이면서, 한 번 생성되면 내부 요소를 변경할 수 없는 불변성(Immutability)의 특징을 가진 튜플(Tuple)의 개념을 학습하였습니다.

리스트(List)와의 차이점을 명확히 파악하여, 프로그램 실행 중 데이터가 오염되지 않아야 하는 안정적인 데이터 관리 상황에서 튜플을 활용하는 방법을 익혔습니다.

함수의 다중 반환값 처리 등 튜플이 실제 개발 환경에서 구조적으로 안전하게 데이터 트래픽을 전달하는 방식을 실습을 통해 이해하였습니다.

Name	Last commit message	Last commit date
..		
hello.py	Add files via upload	3 months ago

WEEK 1

개발자 되기!

- Github 회원가입을 했습니다.
- Python Repository 를 생성하였습니다.
- Python과 VSCode 를 다운로드 했습니다.
- print 명령어를 사용하여 테스트 하였습니다.

—> 개발자가되기위한첫 걸음을 마침내 해내었습니다!!

Commits

History for Python / 0306 / resorce on main

All users

All time

Commits on Mar 6, 2026

Add files via upload



2026964010 authored on Mar 6

Verified

e4aa4e3



[Phase 1] 데이터의 흐름을 통제하다: 변수와 연산 (Week 2)

[Phase 2] 조건의 분기와 순환 로직: 제어문과 반복문 (Week 3-4)

```
1 for i in range(2, 21):
2     is_prime = True
3     for j in range(2, i):
4         if i % j == 0:
5             is_prime = False
6             break
7     if is_prime:
8         print(i, end=" ")
```

효율적인 검증
플래그의 상태 변화

Commits

History for Python / 0327 on `main`

🔍 All users

📅 All time

🔗 Commits on Apr 3, 2026

Add files via upload

🌿 2026964010 authored on Apr 3

Verified

aaecfab



🔗 End of commit history for this file

 All users **All time**

verified 3bd0ab6

verified DocuSign

[Phase 3] 데이터의 군집화와 재사용성: 함수와 리스트 (Week 5-6)

[Phase 4] 파이썬 자료구조 (Week7)

	튜플 (Tuple)	세트 (Set)	딕셔너리 (Dict)
1 생성 형태	(1, 2, 3)	{1, 2, 3}	{"a": 1}
2 순서 보장	보장	없음	보장
3 중복 허용	허용	불가	불가
4 데이터 수정	불변	가능	가능
5 접근 방식	인덱스 [0]	인덱스 [X]	키로 접근
6 핵심 용도	변하지 않는 데이터	중복 제거	키값 쌍 저장

① Insight: 각 자료구조는 다루는 데이터의 '불변성', '순서', '검색 속도' 요구사항에 따라 전략적으로 선택됩니다.

2026964010 Add files via upload

39f8b46 · 2 days ago History

Name	Last commit message	Last commit date
..		
Exercise.txt	Add files via upload	2 days ago
Programming.txt	Add files via upload	2 days ago
acronym.py		2 days ago
address.py		2 days ago
ch_count.py		2 days ago
counter.py		2 days ago
dict.py		last month
email.py		2 days ago
eng_dic.py		2 days ago
otp.py		2 days ago
palin.py		2 days ago
score.py		2 days ago
set.py	Add files via upload	last month
str_ex1.py	Add files via upload	2 days ago
str_ex2.py	Add files via upload	2 days ago
tuple.py	Add files via upload	last month
twitter1.py	Add files via upload	2 days ago
중간참검.도전문제.txt	Add files via upload	2 days ago

Commits

History for Python / 0508 on main

All users

All time

Commits on Jun 9, 2026

Add files via upload

2026964010 authored 2 days ago

Verified

39f8b46



Commits on May 15, 2026

Add files via upload

2026964010 authored last month

Verified

964d610



End of commit history for this file

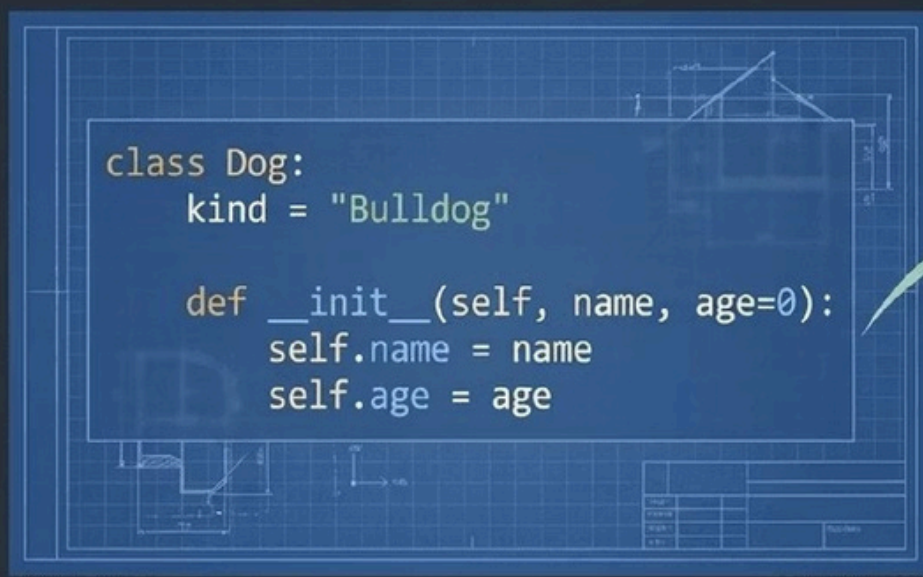
[Phase 4] 객체 지향 프로그래밍의 이해: 클래스 (Week 8)

>_ {}

The Blueprint

```
class Dog:
    kind = "Bulldog"

    def __init__(self, name, age=0):
        self.name = name
        self.age = age
```



>_ {}

Python / 0515 /

2026964010 Add files via upload

Name	Last commit message
...	
account.py	Add files via upload
circle.py	Add files via upload
dice.py	Add files via upload
dog.py	Add files via upload
exercise.종간합정.txt	Add files via upload
programming.txt	Add files via upload
student.py	Add files via upload
student2.py	Add files via upload
vector.py	Add files via upload

Commits

History for Python / 0515 on main

Commits on Jun 9, 2026

Add files via upload

2026964010 authored 2 days ago

End of commit history for this file

클래스는 데이터(속성)와 기능(메서드)을 하나의 캡슐로 묶어내는 강력한 설계도입니다.

비밀번호 생성기 코드1

```
1 import tkinter as tk
2 from tkinter import ttk, messagebox
3 import random
4 import string
5
6 # 문자 집합
7 uppercase = string.ascii_uppercase
8 lowercase = string.ascii_lowercase
9 numbers = string.digits
10 special = "!@#$%^&*?_"
11 all_chars = uppercase + lowercase + numbers + special
12
13 |
14 def get_strength(pw):
15     has_upper = any(c in uppercase for c in pw)
16     has_lower = any(c in lowercase for c in pw)
17     has_number = any(c in numbers for c in pw)
18     has_special = any(c in special for c in pw)
19     length = len(pw)
20
21     if has_upper and has_lower and has_number and has_special and length >= 10:
22         return "보안 강력 ", █ "#2ecc71"
23     elif sum([has_upper, has_lower, has_number, has_special]) >= 3 and length >= 8:
24         return "사용 가능 ", █ "#f39c12"
25     else:
26         return "취약함 ", █ "#e74c3c"
27
```

비밀번호 생성에 필요한 문자 집합을 정의하고, 강도를 측정하는 `get_strength()` 함수를 추가했습니다.

비밀번호 생성기 코드2

```
def generate_password():
    pw = [
        random.choice(uppercase),
        random.choice(lowercase),
        random.choice(numbers),
        random.choice(special)
    ]
    for _ in range(12):
        pw.append(random.choice(all_chars))

    random.shuffle(pw)
    final_pw = ''.join(pw)
    pw_var.set(final_pw)

def on_entry_change(*args):
    """ 글자 입력 시 실시간으로 강도 및 유효성 체크 """
    pw = pw_var.get()
    if not pw:
        strength_label.config(text="")
        return

    # 허용되지 않은 문자 입력 시 실시간 경고
    if any(ch not in all_chars for ch in pw):
        strength_label.config(text="허용되지 않은 문자 포함됨 ❌", fg="red")
        return

    # 실시간 강도 업데이트
    label, color = get_strength(pw)
    strength_label.config(text=f"강도: {label}", fg=color)
```

무작위 비밀번호를 생성하는 `generate_password()` 함수와, 입력 시 실시간으로 강도 및 유효성을 검사하는 `on_entry_change()` 함수를 추가했습니다.

비밀번호 생성기 코드3

```
def copy_to_clipboard():
    pw = pw_var.get()
    if not pw:
        messagebox.showwarning("알림", "비밀번호를 먼저 생성하거나 입력하세요.")
        return
    root.clipboard_clear()
    root.clipboard_append(pw)
    copy_btn.config(text="✓ 복사됨")
    root.after(2000, lambda: copy_btn.config(text="클립보드 복사"))

def toggle_visibility():
    if pw_entry.cget("show") == "*":
        pw_entry.config(show="")
        toggle_btn.config(text="숨기기")
    else:
        pw_entry.config(show="*")
        toggle_btn.config(text="보이기")

# — 메인 윈도우 —
root = tk.Tk()
root.title("비밀번호 생성기")
root.geometry("480x360")
root.resizable(False, False)
root.configure(bg="#2b2b2b")
```

비밀번호를 클립보드에 복사하는
copy_to_clipboard() 함수와 비밀번호
표시/숨김을 전환하는
toggle_visibility() 함수를 추가하고, 메
인 윈도우를 설정했습니다.

비밀번호 생성기 코드4

UI의 색상, 폰트, 버튼 스타일 등 전체적인 디자인 테마를 정의했습니다.

```
# — 스타일 —————
BG      = "#2b2b2b"
SURFACE = "#3c3f41"
ACCENT  = "#4a9eff"
FG      = "#ffffff"
FG2     = "#aaaaaa"
FONT    = ("Consolas", 11)
FONT_LG = ("Consolas", 14, "bold")
FONT_SM = ("Consolas", 9)

btn_style = dict(
    bg=ACCENT, fg=FG, activebackground="#3a8eef", activeforeground=FG,
    relief="flat", cursor="hand2", font=FONT, padx=16, pady=6
)
btn_secondary = dict(
    bg=SURFACE, fg=FG2, activebackground="#4a4d50", activeforeground=FG,
    relief="flat", cursor="hand2", font=FONT, padx=16, pady=6
)

# — 타이틀 —————
tk.Label(root, text="🔒 비밀번호 생성기", font=("Consolas", 16, "bold"), bg=BG, fg=FG).pack(pady=(24, 4))
tk.Label(root, text="랜덤 생성 또는 직접 입력 후 강도를 확인하세요", font=FONT_SM, bg=BG, fg=FG2).pack()

# — 구분선 —————
tk.Frame(root, height=1, bg="#444").pack(fill="x", padx=24, pady=14)
```

비밀번호 생성기 코드5

```
114 # — 비밀번호 입력란 —————
115 pw_var = tk.StringVar()
116 pw_var.trace_add("write", on_entry_change)
117
118 frame_entry = tk.Frame(root, bg=SURFACE, bd=0)
119 frame_entry.pack(padx=24, fill="x")
120
121 pw_entry = tk.Entry(frame_entry, textvariable=pw_var, font=FONT_LG,
122                    bg=SURFACE, fg=FG, insertbackground=FG,
123                    relief="flat", show="*", bd=0)
124 pw_entry.pack(side="left", fill="x", expand=True, pady=10, padx=(12, 0))
125
126 toggle_btn = tk.Button(frame_entry, text="보이기", command=toggle_visibility,
127                       bg=SURFACE, fg=FG2, activebackground=□"#4a4d50", activeforeground=FG,
128                       relief="flat", cursor="hand2", font=FONT, padx=10, pady=4)
129 toggle_btn.pack(side="right", padx=6, pady=4)
130
131 # — 강도 표시 —————
132 strength_label = tk.Label(root, text="", font=("Consolas", 11, "bold"), bg=BG, fg=FG)
133 strength_label.pack(pady=(10, 2))
134
135 # — 버튼 영역 (2개 버튼으로 균형 재배치) —————
136 frame_btns = tk.Frame(root, bg=BG)
137 frame_btns.pack(pady=14)
138
139 tk.Button(frame_btns, text="🎲 랜덤 생성", command=generate_password, **btn_style).grid(row=0, column=0, padx=8)
140 copy_btn = tk.Button(frame_btns, text="클립보드 복사", command=copy_to_clipboard, **btn_secondary)
141 copy_btn.grid(row=0, column=1, padx=8)
142
143 # — 허용 문자 안내 —————
144 tk.Label(root, text=f"허용 문자:  A-Z  a-z  0-9  {special}", font=FONT_SM, bg=BG, fg=□"#666666").pack(pady=(8, 0))
145
146 root.mainloop()
```

비밀번호 입력창, 강도 표시 레이블, 생성/복사 버튼 등 실제 UI 위젯을 배치했습니다.

User Journey & UI States (실행 데모)

State 1: 자동 생성 및 강력한 보안

 비밀번호 생성기

랜덤 생성 또는 직접 입력 후 강도를 확인하세요

gc9\$ub@GY3ZW66Nc

숨기기

강도: 보안 강력

 랜덤 생성

클립보드 복사

허용 문자: A-Z a-z 0-9 |@!%*~?_

State 2: 사용자 직접 입력 및 경고

 비밀번호 생성기

랜덤 생성 또는 직접 입력 후 강도를 확인하세요

gc9\$ub@

숨기기

강도: 취약함

 랜덤 생성

클립보드 복사

허용 문자: A-Z a-z 0-9 |@!%*~?_

State 3: 예외 처리 (허용되지 않은 문자)

 비밀번호 생성기

랜덤 생성 또는 직접 입력 후 강도를 확인하세요

gc9\$ub@/

숨기기

허용되지 않은 문자 포함됨 ✖

 랜덤 생성

클립보드 복사

허용 문자: A-Z a-z 0-9 |@!%*~?_

Robust Edge-Case Handling

코드가 완벽하게 작동하여 생성, 입력 수정, 마스킹(***) 처리, 그리고 실시간 유효성 검사까지 사용자의 모든 상호작용(Journey)에 즉각적으로 반응하는 높은 완성도의 애플리케이션을 완성했습니다.

마무리 감상

코딩, 프로그래밍 코드가 저에게는 외계어 같았습니다.

하지만 프로그래밍도 언어도 다른 언어공부할 때 처럼 계속 읽고 쓰니 독해력이 느는것을 체감할 수 있었습니다.

영어도 꾸준히 공부해야 늘 듯, 코딩도 벼락치기가 아닌 꾸준함으로 접해야 강해질 수 있다는 결론에 다다랐습니다.

누구나 언어공부를 꾸준히 하면 능통해지듯 저도 꾸준히 코딩을 보고 쓴다면 충분히 멋진 빌더가 될 수 있다고 생각합니다.

예상 점수

Github 정리 15/15

프로그램 프로젝트 15/15

총합 30/30 !

감사합니다